

**LOCAL NOTIFICATIONS**

**SOCIAL SHARING**

**DEMO**





1

# LOCAL NOTIFICATIONS

- Is a message that app displays outside your app's UI to provide the user with reminders, communication from other people, or other timely information from your app
- Notifications type
  - Local Notifications
  - Push Notification

# LOCAL NOTIFICATIONS

- Are scheduled, handle and triggered in the same device and are triggered inside the device
- Example
  - Reminder App to show notifications

# PUSH NOTIFICATION

- Not triggered inside the app but are send due to some action outside the app
- Action
  - Database
  - App updates version
  - Messages from other users or the admins
- Are sent remotely to one or many users simultaneousl
- Example
  - E-Commerce
  - Email
  - Marketing Notifiactions

# PUSH NOTIFICATION

- Notification gateways – service
- Push token

# NOTIFICATION GATEWAYS - SERVICE

- Expose a Web API for sending push notifications to users through HTTP requests
  - FCM (Firebase Cloud Messaging) for Android devices
  - APNs (Apple Push Notification service) for iOS devices

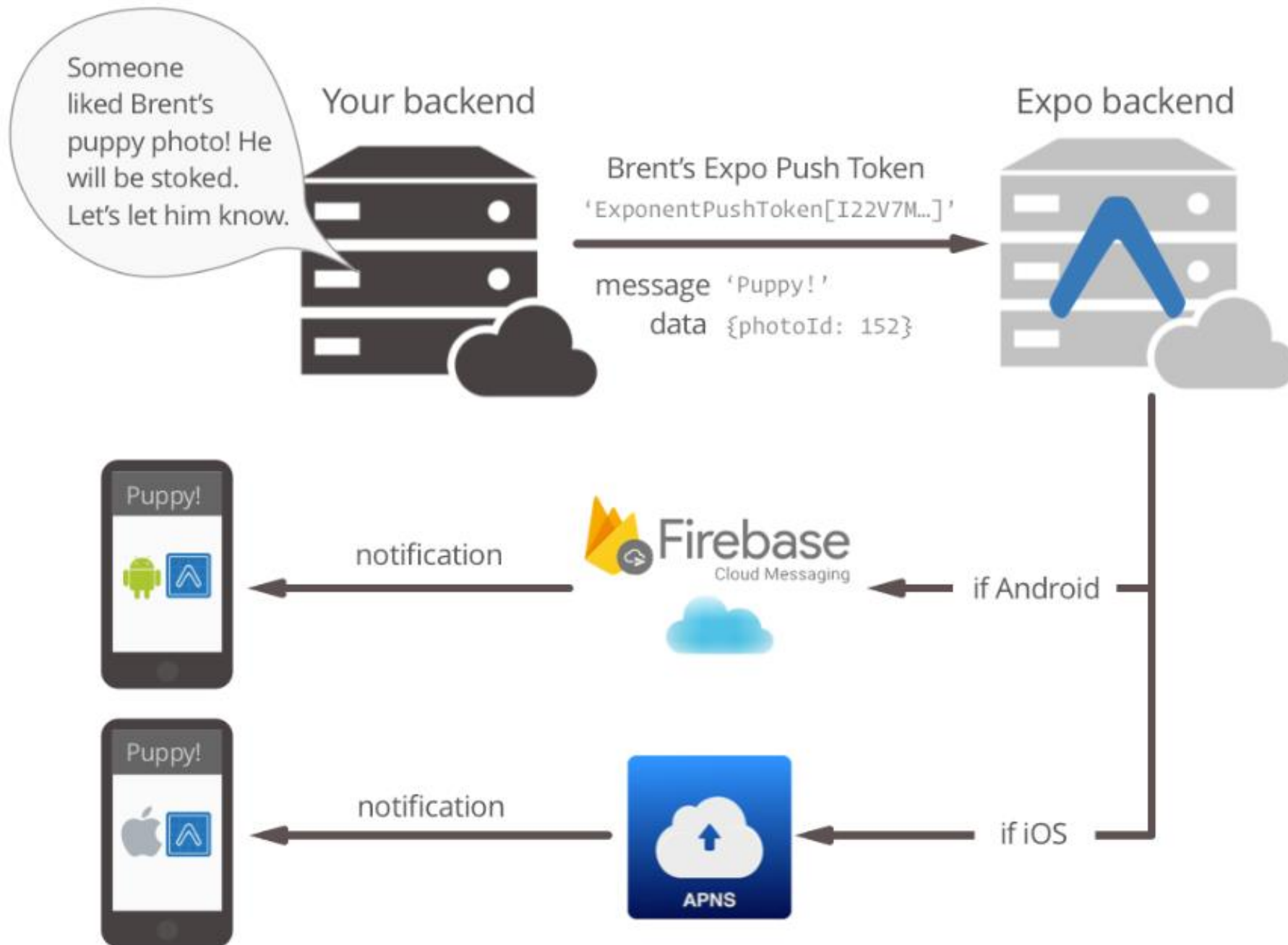
# PUSH TOKENS

- A unique key that identifies an app installed in a specific device
- Allow delivering the messages only to the intended app-device combination
- Issued by
  - Apple push notification gateways
  - Google push notification gateways



# EXPO NOTIFICATIONS

- Provides an API to fetch push notification tokens and to present, schedule, receive and respond to notifications
- Features
  - Schedule a one-off notification for a specific date, or some time from now
  - Schedule a notification repeating in some time interval (or a calendar date match on iOS)
  - Get and set application badge icon number
  - Fetch a native device push token so you can send push notifications with FCM and APNS
  - Fetch an Expo push token so you can send push notifications with Expo
  - Listen to incoming notifications in the foreground and background



# INSTALLATION

- Library
  - npm install **expo-notifications**
  - npm install **expo-device**
  - npm install **expo-constants**
- Usage

```
import * as Notifications from 'expo-notifications';
```

```
import * as Device from 'expo-device';
```

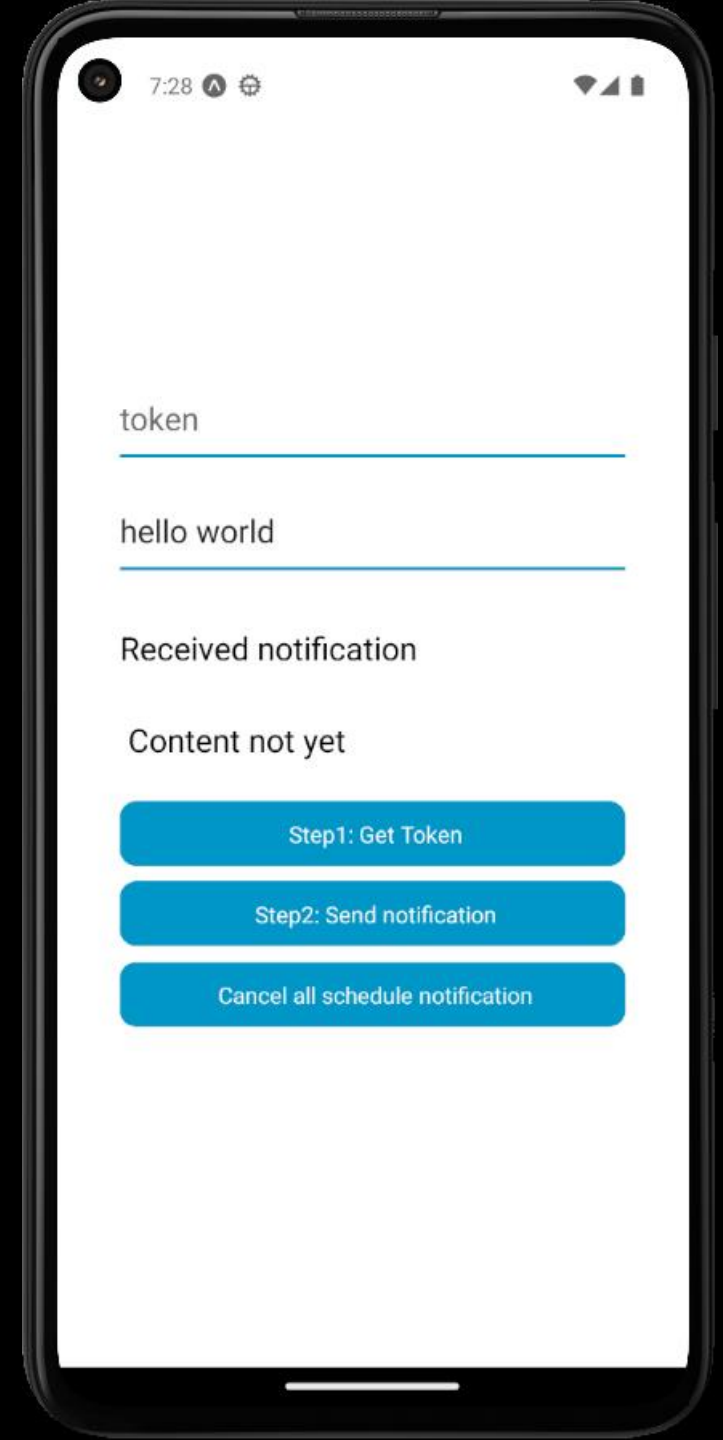
```
import Constants from 'expo-constants';
```

# Sample - push notifications

## **Initialize the Notification Handler**

```
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import Constants from 'expo-constants';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: false,
    shouldSetBadge: false,
  }),
});
```



**Adds the notification listener and subscriber**

```
const Demo = ({ }) =>
{
  const [token, setToken] = useState("");
  const [msg, setMsg] = useState("hello world");
  const [notification, setNotification] = useState();
  const notificationListener = useRef();
  const responseListener = useRef();

  useEffect(() =>
  {
    getToken().then(token => setToken(token));
    //Update the app on notification received.
    notificationListener.current =
      Notifications.addNotificationReceivedListener(notification => {
        setNotification(notification);
        console.log("Notification is received", notification)
      });
    //Update the app on notification clicked
    responseListener.current =
      Notifications.addNotificationResponseReceivedListener(response => {
        console.log("Notification is clicked", response)
      });

    return () =>
    {
      Notifications.removeNotificationSubscription(notificationListener.current);
      Notifications.removeNotificationSubscription(responseListener.current);
    };
  }, []);
```



**Send notification**

```

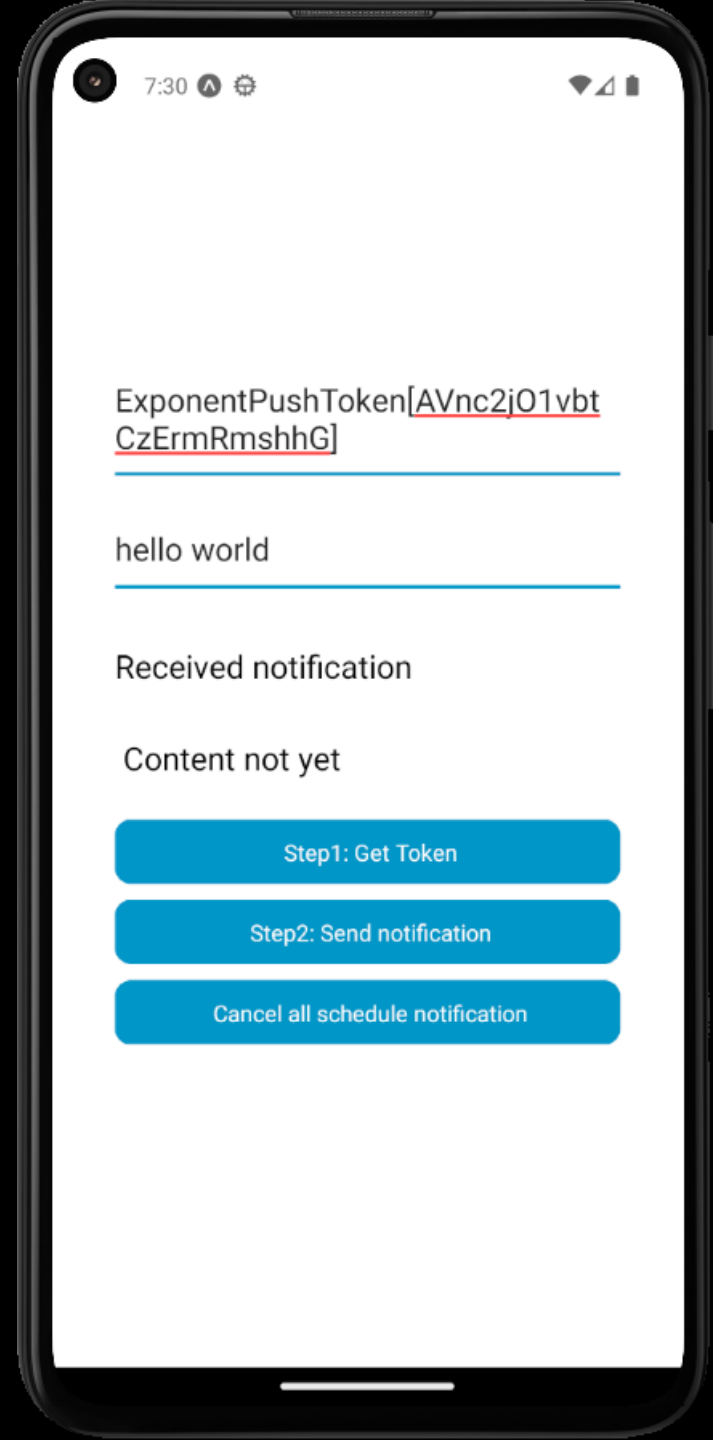
const getToken = async () =>
{
  let status;
  let result;
  status = await Notifications.getPermissionsAsync();
  if (status !== "granted")
  {
    result = await Notifications.requestPermissionsAsync();
    console.log(result);
  }
  if (result.status !== "granted")
  {
    alert("Failed to get push token for push notification!");
    return;
  } else
  {
    const projectId = Constants.expoConfig.extra.eas.projectId;

    if (!projectId)
    {
      throw new Error('Project ID not found!!!');
    }
    let expoPushToken = (await
Notifications.getExpoPushTokenAsync(projectId)).data;

    setToken(expoPushToken);
    console.log(expoPushToken);

  }
  return token;
}

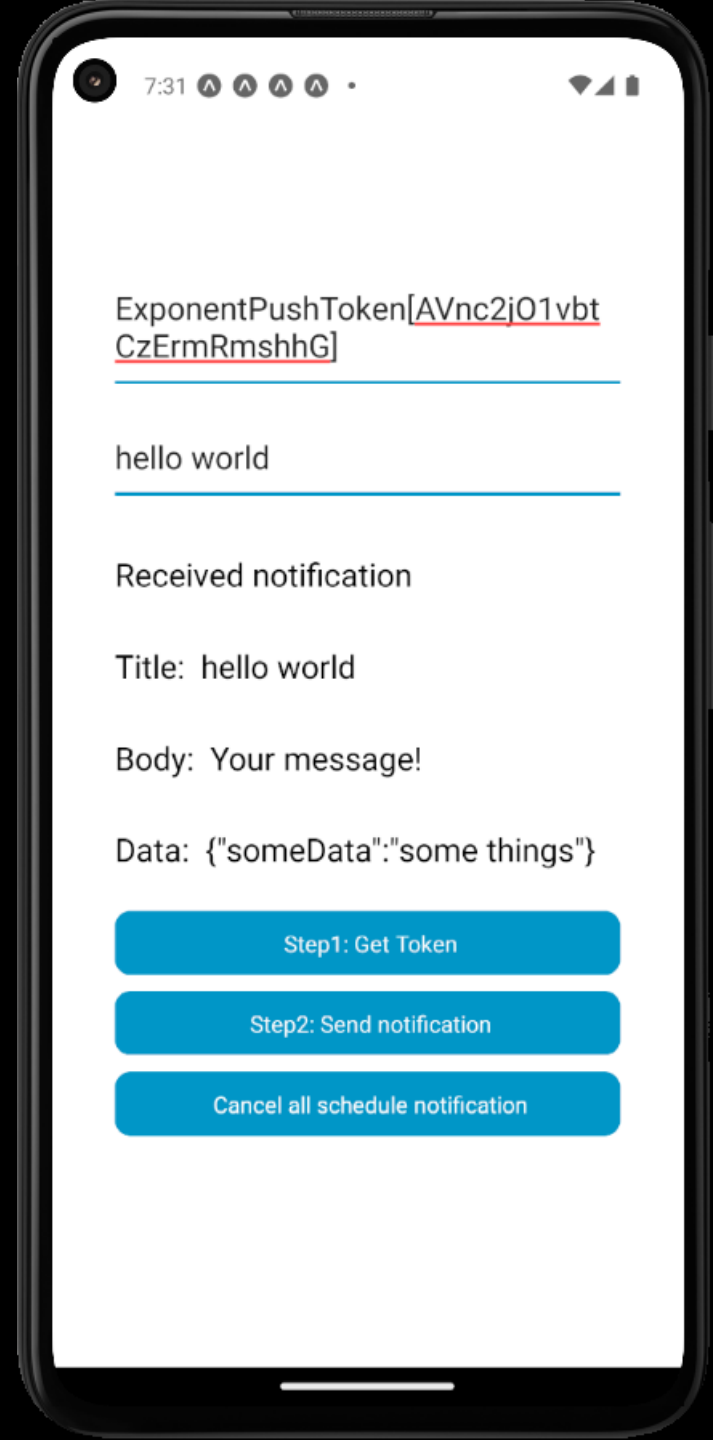
```



```
const sendPushNotification = async (expoPushToken) =>
{

  const message = {
    to: [expoPushToken, expoPushToken],
    title: msg,
    body: 'Your message!',
    data: { someData: 'some things' },
  };

  await fetch('https://exp.host/--/api/v2/push/send', {
    method: 'POST',
    headers: {
      Accept: 'application/json',
      'Accept-encoding': 'gzip, deflate',
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(message),
  });
}
```



```
const scheduleSendNotification = async () => {  
  const id = await Notifications.scheduleNotificationAsync({  
    content: {  
      title: "You have a new message",  
      body: 'Here is your message',  
      data: { customData: message },  
    },  
    trigger: { seconds: 60, repeats: true },  
  
  });  
  console.log("Your message id", id)  
  
}
```

# EAS build

- EAS Build allows you to build a ready-to-submit binary of your app for the Google Play Store or Apple App Store
- Create Expo account
  - <https://expo.dev/signup>
- Download and install Git
  - <https://git-scm.com/download/win>
- Configure an Android or an iOS project for EAS Build
  - `npm install -g eas-cli`
  - `eas login`
  - `eas build:configure`

JS App.js

{ } app.json



Bai11 > { } app.json > ...

```
2      "expo": {
26      "web": {
28      },
29      "extra": {
30        "eas": {
31          "projectId": "90adf10b-9433-4101-ba07-d55a72eec83f"
32        }
33      }
34    }
35  }
36
```



2

## SOCIAL SHARING

# Usage

```
import {Share} from 'react-native'
```

```
static share(content: ShareContent, options?: ShareOptions)
```

| NAME                           | TYPE   | DESCRIPTION                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| content<br><div>Required</div> | object | <div>message - a message to share</div> <div>url - a URL to share </div> <div>title - title of the message </div>                                                                                                                                                                                                                                                                                                                                                                          |
|                                |        | At least one of url and message is required.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| options                        | object | <div>dialogTitle </div> <div>excludedActivityTypes </div> <div>subject - a subject to share via email </div> <div>tintColor </div> <div>anchor - the node to which the action sheet should be anchored (used for iPad) </div> |



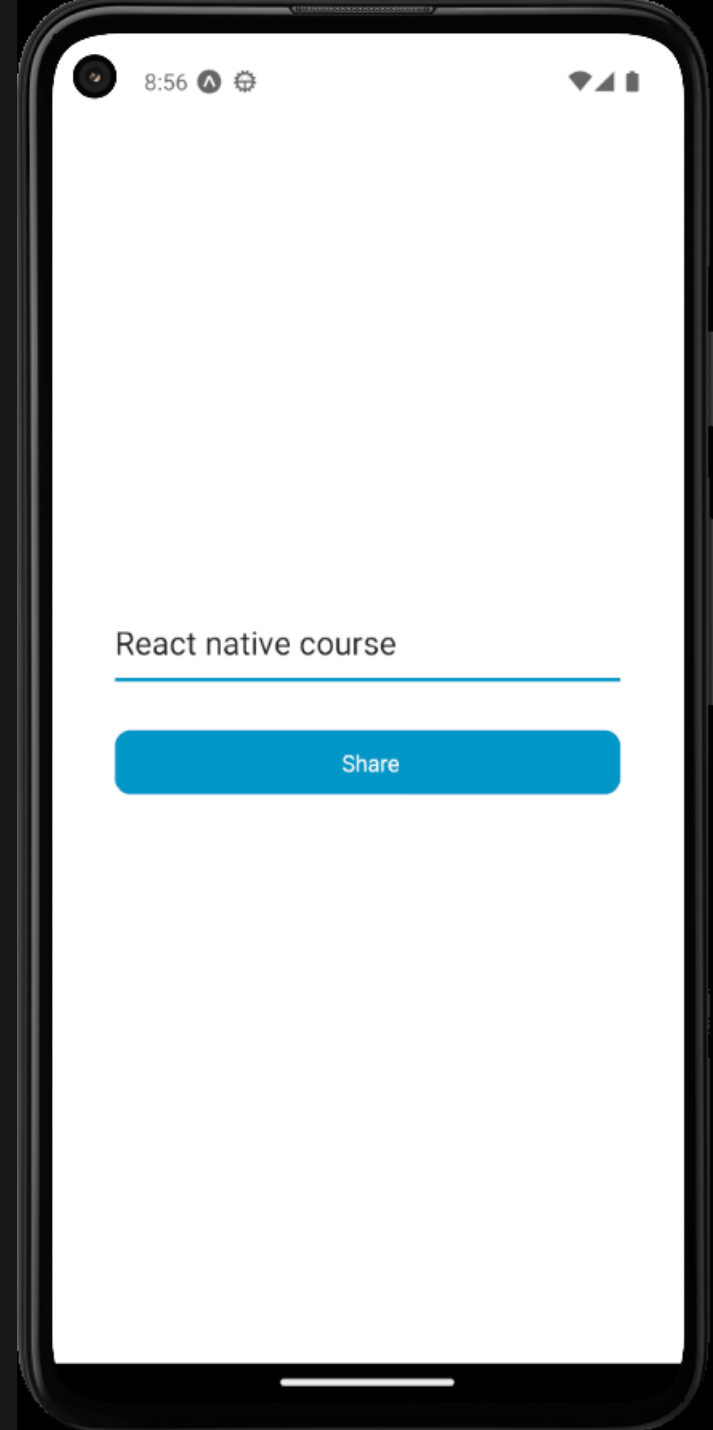
```

<View style={styles.container}>
  <View style={styles.formContainer}>
    <View style={styles.inputContainer}>
      <TextInput
        placeholder="Message"
        style={styles.inputText} value={msg}
        multiline={true} />
    </View>
    <TouchableOpacity style={styles.btn}
      onPress={() => onShare()}>

      <Text style={styles.btnText}> Share</Text>

    </TouchableOpacity>
  </View>
</View>

```



```
const [msg, setMsg] = useState("React native course")
const onShare = async () =>
{
  try
  {
    const result = await Share.share({
      message: msg,
      title: 'Share message',
    });

    if (result.action === Share.sharedAction)
    {
      if (result.activityType)
      {
        console.log('shared', result.activityType)
      } else
      {
        // shared
        console.log("shared done")
      }
    } else if (result.action === Share.dismissedAction)
    {
      // dismissed
      console.log('dismissed')
    }
  } catch (error)
  {
    Alert.alert(error.message);
  }
};
```

